Well-Well — Low-Level Design

Asset Intelligence Copilot · FlytBase challenge, Hacktoberfest 2026 Hack Day Pune · Gemma 4 · version 1.0, 17 Oct 2026

> **Design principle.** The language model decides *what to look up and how to say it*; deterministic code decides *what is true*. Every number shown to a user is computed by SQL/Python from the records and every claim carries the IDs of the records and document sections it came from.

1. Component view

```
Streamlit UI (ui/app.py)
  user picker · conversation list · chat · inline visuals · citation chips · trust panel · 👍/👎 · memory drawer
        │  services.copilot.ask(user_id, question, conversation_id, provider, on_step)
        ▼
services/copilot.py ── CopilotService
  check_input()  →  load history (own conversation only)  →  build_system()  →  Agent.run()  →  persist
        │
        ▼
harness/agent.py ── Agent                                  harness/llm.py ── LLM gateway
  rounds ≤ 6 · budget 120 s · parallel calls ≤ 4    ◀──▶    GemmaOnGemini (26B A4B → 31B fallback)
  extract_json() · _finish() · _choose_visual()             OpenAICompatible (Ollama / OpenRouter)
        │ ToolBox.run(name, args)
        ▼
harness/tools.py ── ToolBox (18 tools, typed specs)       harness/guardrails.py · harness/sql_guard.py
        │
        ├─ analytics/facts.py        Facts       (status, spills, downtime, aggregate, repairs, RCA, details)
        ├─ analytics/compliance.py   Compliance  (SR-01/02/03, RC-01/02/04, pending actions)
        ├─ analytics/documents.py    BM25 search over manual / SOP / report sections
        ├─ analytics/periods.py      period strings → closed date ranges
        └─ memory/feedback.py        Feedback (corrections, notes, rules, claims) · PrivateMemory (recall)
        │
        ▼
db/  SQLAlchemy models + views.sql  ──  SQLite (data/wellwell.db), Postgres-compatible models
        ▲
ingestion/loader.py ── DatasetLoader  (+ ingestion/documents.py extractors, ingestion/normalize.py)
```

Module responsibilities

Module	Key types / functions	Responsibility
`config.py`	`Settings, get_settings()`	.env driven settings: provider, Gemma model ids, DB URL, budgets, "today".
`db/base.py`	`Base, enum_column(), LoadedFromMixin`	Declarative base; enums stored as TEXT + CHECK; every dataset row records dataset + source file.
`db/models/*`	`fleet.py, records.py, memory.py`	Tables in §3.
`db/session.py`	`get_engine(), get_read_only_engine(), session_scope(), create_schema(), create_views()`	Read-write engine (WAL) and a separate read-only engine (`mode=ro` + `PRAGMA query_only`) for agent SQL.
`db/views.sql`	`inspection_facts, latest_inspection`	Corrections applied, temperature margin, producing flag, effective spills (§4).

`ingestion/loader.py`	`DatasetLoader.run() → LoadReport`	Idempotent upsert of a dataset folder; documents; corrections from notes.
`ingestion/documents.py`	`extract_manual_rules()`, `extract_report_facts()`, `split_sections()`	Rules from manuals; containment, class, compliance table from reports.
`analytics/facts.py`	`Facts`, `spill_class()`	All read tools except compliance; explanations of stops; causes of downtime.
`analytics/compliance.py`	`Compliance.check()`	SOP deadline checks with citations.
`analytics/result.py`	`ToolResult`, `cite()`	Uniform tool output (§6).
`memory/feedback.py`	`Feedback`, `PrivateMemory`	Writes shared memory under the signed-in user; private recall.
`harness/*`	`Agent`, `ToolBox`, `SPECS`, `build_system()`, `guard_sql()`, `check_input()`	Agent loop, tool registry, prompts, guardrails.
`services/copilot.py`	`ask()`, `start_conversation()`, `conversation_history()`, `conversation_messages()`, `rate_answer()`	Entry point for UI and CLI; persistence; privacy checks.
`cli.py`	`wellwell load | ask`	Load datasets, ask from the terminal.

2. Key flows

2.1 Ask a question

```
User ─► UI ─► ask(user, q, conv)
              ├─ check_input(q)                       blocked? → polite refusal, nothing stored
              ├─ load User, Conversation (must belong to user) and last 6 messages
              ├─ system = build_system(today, user, team) + tool catalogue
              └─ Agent.run(q, history)
                   loop (≤ 6 rounds, ≤ 120 s):
                     prompt = history + question + "Step n tool(args) → ToolResult.for_model()" …
                     reply  = LLM.complete(system, prompt)     (Gemma 4; thought parts dropped)
                     action = extract_json(reply)              (fences, prose, trailing commas tolerated)
                     ├─ {"calls":[…]} → ToolBox.run() each (errors become observations)
                     └─ {"final":{answer, visual, citations}}
                          no tool yet & factual?  → "call a tool first", continue
                          _finish(): drop unknown citations · flag ungrounded numbers
                                     enforce requested visual · pick tool rows for the visual
              ├─ persist Message(user), Message(assistant + citations + trust + visual meta), ToolRun × n
              └─ CopilotReply{answer, visual{kind, rows|card}, citations, steps, trust, model, message_id}
```

2.2 Correction (e.g. "the 15 Oct reading for WELL-0164 was 41.0 °C")

```
Agent ─► record_correction(well, on_date, field=wellhead_temp_c, value=41.0, reason, kind=raw_data)
         ├─ inspection_detail() resolves INS-0164-0205; snapshot "before"
         ├─ supersede older active corrections for (inspection, field); note if another user disagreed
         ├─ INSERT corrections(original_value=24.0, corrected_value=41.0, author=signed-in user, reason)
         ├─ inspection_facts now yields 41.0 − 19.9 = 21.1 °C ≥ 15 → producing   ("after" snapshot)
         ├─ inspector note "Pump jack not moving." matches NOT_RUNNING_HINTS → disagreement returned
         └─ for spill fields: re-run Compliance._spill_checks() → SR-01/SR-02/SR-03 effects
Every later answer, for every user, reads inspection_facts → sees the correction and who made it.
```

2.3 Claim (e.g. Tyler: "WELL-0175 is running fine")

```
record_claim(well, claimed_state=producing, statement)
  ├─ well_status() → latest inspection INS-0175-… by Jessica Reed, 0.4 °C above air → not producing
  ├─ status = contradicted_by_data (kept as unconfirmed) · verdict text with reading + inspector
  ├─ other statements: earlier claims/notes with the opposite state (e.g. Michael Turner's shut-in note)
  └─ INSERT notes(origin=copilot, text + [unconfirmed]) and claims(...)
```

2.4 Load a dataset

```
wellwell load <folder> --dataset sample|test [--reset]
  create_schema() → fleet/areas.json, users.json → teams, areas, users
  fleet/well-types/*.md → documents + extract_manual_rules() → well_type_rules
  fleet/sops/*.md → documents
  assets/WELL-*/ metadata.json → wells, equipment
                 inspections.jsonl → inspections            (INSERT OR REPLACE, 2,000-row batches)
                 maintenance.jsonl + report_file → maintenance + documents + report_compliance
                 regulatory-events.jsonl → regulatory_events
                 context.jsonl → notes
  formal correction notes → corrections ("not a spill" → spill_detected=false; "x bbl" → volume)
  create_views() · LoadReport{counts, warnings}
```

3. Data model

Primary keys are the dataset's own IDs (INS-0001-0203, MNT-0001-032, REG-0001-002, CTX-0001-003), so citations need no translation and a second dataset merges by key. Copilot-created records use COR-nnnn, FB-nnnn, CLM-nnnn, RULE-nnnn; documents use OM-T200, SOP-SR, RPT-MNT-…

Dataset (as published)

areas

Column	Type	Keys
area_id	VARCHAR(40)	PK
name	VARCHAR(200)	
team_id	VARCHAR(40)	FK → teams.team_id
center_lat	FLOAT	
center_lon	FLOAT	
dataset	VARCHAR(40)	
source_file	VARCHAR(300)	

teams

Column	Type	Keys
team_id	VARCHAR(40)	PK
name	VARCHAR(200)	
lead_user_id	VARCHAR(40)	
dataset	VARCHAR(40)	
source_file	VARCHAR(300)	

users

Column	Type	Keys
user_id	VARCHAR(40)	PK
name	VARCHAR(200)	
role	VARCHAR(40)	
team_id	VARCHAR(40)	
dataset	VARCHAR(40)	
source_file	VARCHAR(300)	

wells

Column	Type	Keys
asset_id	VARCHAR(40)	PK
name	VARCHAR(200)	
asset_category	VARCHAR(40)	
well_type	VARCHAR(20)	idx
area_id	VARCHAR(40)	FK → areas.area_id, idx
team_id	VARCHAR(40)	
in_charge_user_id	VARCHAR(40)	
lat	FLOAT	
lon	FLOAT	
installed_on	DATE	
recorded_status	VARCHAR(40)	
last_inspection_id	VARCHAR(40)	
last_inspected_at	DATETIME	
dataset	VARCHAR(40)	
source_file	VARCHAR(300)	

equipment

Column	Type	Keys
equipment_id	VARCHAR(60)	PK
asset_id	VARCHAR(40)	FK → wells.asset_id, idx
kind	VARCHAR(40)	
model	VARCHAR(100)	
dataset	VARCHAR(40)	
source_file	VARCHAR(300)	

inspections

Column	Type	Keys
inspection_id	VARCHAR(40)	PK
asset_id	VARCHAR(40)	FK → wells.asset_id
inspected_at	DATETIME	
inspection_date	DATE	idx
inspector_user_id	VARCHAR(40)	
method	VARCHAR(40)	
ambient_temp_c	FLOAT	
wellhead_temp_c	FLOAT	
spill_detected	BOOLEAN	
spill_area_pct	FLOAT	
spill_est_volume_bbl	FLOAT	
evidence_images	JSON	
notes	TEXT	
dataset	VARCHAR(40)	
source_file	VARCHAR(300)	

maintenance

Column	Type	Keys
maintenance_id	VARCHAR(40)	PK
asset_id	VARCHAR(40)	FK → wells.asset_id
started_at	DATETIME	
completed_at	DATETIME	
maintenance_type	VARCHAR(40)	
equipment_kind	VARCHAR(40)	
issue	TEXT	
action_taken	TEXT	
technician_user_id	VARCHAR(40)	
contractor	VARCHAR(200)	
downtime_hours	FLOAT	
cost_labor_usd	FLOAT	
cost_parts_usd	FLOAT	
cost_total_usd	FLOAT	
triggered_by_inspection_id	VARCHAR(40)	idx
report_file	VARCHAR(300)	
report_doc_id	VARCHAR(80)	
report_containment_hours	FLOAT	
report_spill_class	VARCHAR(40)	
report_written_on	DATE	
dataset	VARCHAR(40)	
source_file	VARCHAR(300)	

regulatory_events

Column	Type	Keys
event_id	VARCHAR(40)	PK
asset_id	VARCHAR(40)	FK → wells.asset_id, idx
date	DATE	
agency	VARCHAR(200)	
event_type	VARCHAR(40)	
protocol_id	VARCHAR(20)	
related_inspection_id	VARCHAR(40)	idx
hours_to_report	FLOAT	
detail	TEXT	
fine_usd	FLOAT	
status	VARCHAR(20)	
dataset	VARCHAR(40)	
source_file	VARCHAR(300)	

notes

Column	Type	Keys
note_id	VARCHAR(40)	PK
origin	VARCHAR(40)	
scope	VARCHAR(40)	
asset_id	VARCHAR(40)	FK → wells.asset_id
well_type	VARCHAR(20)	
created_at	DATETIME	
author_user_id	VARCHAR(40)	
note_type	VARCHAR(40)	
inspection_id	VARCHAR(40)	idx
text	TEXT	
valid_from	DATE	
valid_to	DATE	
dataset	VARCHAR(40)	
source_file	VARCHAR(300)	

Extracted at load time

well_type_rules

Column	Type	Keys
well_type	VARCHAR(20)	PK
manual_doc_id	VARCHAR(40)	
producing_min_delta_c	FLOAT	
producing_rule_ref	VARCHAR(60)	
temp_alarm_max_c	FLOAT	
rca_min_corrective	INTEGER	
rca_window_days	INTEGER	
rca_rule_ref	VARCHAR(60)	
notes	TEXT	

report_compliance

Column	Type	Keys
id	INTEGER	PK
maintenance_id	VARCHAR(40)	idx
asset_id	VARCHAR(40)	idx
protocol_id	VARCHAR(20)	
result	VARCHAR(40)	
met	BOOLEAN	
detail	TEXT	

documents

Column	Type	Keys
doc_id	VARCHAR(80)	PK
kind	VARCHAR(40)	
title	VARCHAR(300)	
asset_id	VARCHAR(40)	idx
maintenance_id	VARCHAR(40)	
well_type	VARCHAR(20)	
path	VARCHAR(400)	
text	TEXT	
dataset	VARCHAR(40)	

document_chunks

Column	Type	Keys
id	INTEGER	PK
doc_id	VARCHAR(80)	FK → documents.doc_id, idx
chunk_index	INTEGER	
section	VARCHAR(300)	
text	TEXT	
embedding	BLOB	

Memory and feedback

corrections

Column	Type	Keys
correction_id	VARCHAR(40)	PK
kind	VARCHAR(40)	
scope	VARCHAR(40)	
asset_id	VARCHAR(40)	idx
well_type	VARCHAR(20)	
target_table	VARCHAR(40)	
target_id	VARCHAR(60)	
field	VARCHAR(60)	
original_value	TEXT	
corrected_value	TEXT	
reason	TEXT	
author_user_id	VARCHAR(40)	
source_note_id	VARCHAR(40)	
status	VARCHAR(40)	
created_at	DATETIME	

rule_overrides

Column	Type	Keys
override_id	VARCHAR(40)	PK
scope	VARCHAR(40)	
well_type	VARCHAR(20)	
rule	VARCHAR(60)	
equipment_kind	VARCHAR(40)	
value	DOUBLE	
manual_value	DOUBLE	
statement	TEXT	
author_user_id	VARCHAR(40)	
status	VARCHAR(40)	
created_at	DATETIME	

claims

Column	Type	Keys
claim_id	VARCHAR(40)	PK
asset_id	VARCHAR(40)	idx
author_user_id	VARCHAR(40)	
statement	TEXT	
claimed_state	VARCHAR(40)	
effective_from	DATE	
status	VARCHAR(40)	
evidence	JSON	
note_id	VARCHAR(40)	
created_at	DATETIME	

answer_feedback

Column	Type	Keys
id	INTEGER	PK
message_id	INTEGER	FK → messages.message_id, idx
user_id	VARCHAR(40)	idx
rating	VARCHAR(10)	
comment	TEXT	
created_at	DATETIME	

Conversations (private per user)

conversations

Column	Type	Keys
conversation_id	INTEGER	PK
user_id	VARCHAR(40)	idx
title	VARCHAR(300)	
started_on	DATE	
created_at	DATETIME	

messages

Column	Type	Keys
message_id	INTEGER	PK
conversation_id	INTEGER	FK → conversations.conversation_id, idx
user_id	VARCHAR(40)	idx
role	VARCHAR(20)	
content	TEXT	
answer	JSON	
asset_ids	JSON	
sent_on	DATE	
model_id	VARCHAR(100)	
latency_ms	INTEGER	
created_at	DATETIME	

tool_runs

Column	Type	Keys
id	INTEGER	PK
message_id	INTEGER	FK → messages.message_id, idx
step	INTEGER	
tool	VARCHAR(60)	
args	JSON	
result	JSON	
error	TEXT	
latency_ms	INTEGER	
created_at	DATETIME	

4. Derived views

inspection_facts

```
for each inspection i:
  c_wellhead, c_ambient, c_spill, c_volume = latest ACTIVE correction value per field (or NULL)
  eff_wellhead = COALESCE(c_wellhead, i.wellhead_temp_c)        eff_ambient = COALESCE(c_ambient, i.ambient_temp_c)
  threshold    = latest ACTIVE rule_overrides(producing_min_delta_c) for the well type or fleet,
                 else well_type_rules.producing_min_delta_c (15 °C T200, 20 °C T350 — read from the manuals)
  temp_delta_c = ROUND(eff_wellhead − eff_ambient, 1)
  producing    = (eff_wellhead − eff_ambient) ≥ threshold
  eff_spill    = c_spill if present else i.spill_detected
  eff_volume   = c_volume or i.spill_est_volume_bbl, only when eff_spill
  has_correction, area_name, lat, lon, well_type carried along
latest_inspection = inspection_facts rows at each well's MAX(inspected_at)
```

5. Algorithms

Fact	Definition used in code
Producing now	`latest_inspection.producing`. Recorded status is shown as out of date when it disagrees.
Downtime	7 × count of `inspection_facts` rows with producing = 0 and inspection_date in [start, end]. Missed inspections add nothing. `maintenance.downtime_hours` is never used.
Cause of a stop	Repair triggered by that inspection, else a repair whose [started, completed] covers it or starts within 3 days, else a note valid on that date; otherwise "no recorded cause".
Reason not producing	Open repairs at the latest inspection + notes covering the date (valid_from ≤ d+7 and valid_to ≥ d−7, or open-ended and created within 45 days) + copilot notes and claims.
Periods	Rolling windows end yesterday (last full day of data) and count back: last_4_months = 16 Jun–16 Oct 2026; last_90_days = 18 Jul–16 Oct. Calendar years and months are closed ranges. A repair belongs to a period by its start date.
Spill class	<1 minor · 1–<5 moderate · 5–<25 reportable · ≥25 major (SOP-SR §3), on the effective volume.
SR-01	Reportable/major: spill_report_filed for the inspection with hours_to_report ≤ 24 (≤ 2 major); none and deadline passed → breach ("overdue"). Linked notice of violation adds its fine and status.
SR-02	`maintenance.report_containment_hours` of the repair triggered by the spill inspection > 4 → breach.
SR-03	Reportable/major: closed site_inspection for the inspection → met; else due = discovery + 60 days; past due → breach, otherwise pending.
RC-04	Notice of violation needs nov_response_filed within 30 days; none and past due → breach.
RC-02 / RC-01	"Annual flowline integrity test" repair in each calendar year; gaps > 14 days between inspections.
RCA due	Corrective repairs per (well, equipment kind) with start in (today−365, today] ≥ threshold; threshold from the manual or the newest active team rule for that well type and equipment.
Report class mismatch	Repair report's spill class differs from the class of the effective volume → "review" item (the report's reporting assessment no longer holds).

6. Tool contracts

Every tool returns `ToolResult{summary, rows[], card{}, citations[], visual{kind,…}, rules_applied[], warnings[]}`. The model sees `for_model()` (rows capped at 40, no visual); the UI draws from `rows/card` of the step the answer points to.

Tool	Mode	Purpose	Arguments
`well_status`	read	One well right now: producing or not (from the latest inspection and the manual rule), last inspection and inspector, recorded status and whether it is out of date, reasons, notes, corrections, user claims.	`well`
`wells_not_producing`	read	Wells not producing at their latest inspection, with the reason for each (open repairs, notes) or 'no recorded reason'.	`area`, `well_type`, `wells`
`temperature_history`	read	Wellhead vs air temperature readings for one well, with producing flag per reading. Default the last 13 weekly readings.	`well`, `weeks`, `period`
`spills`	read	Real spills (corrected detections removed) for a well or area in a period: date, volume, class, location, regulator report status. Also lists detections excluded by corrections.	`well`, `area`, `period`, `min_volume_bbl`, `reportable_only`
`downtime`	read	Downtime days per well (7 days per weekly inspection not producing) in a period, with the cause of each not-producing inspection. Use top_n for rankings.	`wells`, `area`, `well_type`, `period`, `top_n`
`aggregate`	read	Counts or sums across one or two dimensions, for bar charts and heatmaps. metric: spill_count | spill_volume_bbl | downtime_days | repair_cost_usd | repair_count | corrective_repair_count | fines_usd. rows_by / columns_by: area | well | well_type | month | year | equipment_kind.	`metric`, `rows_by`, `columns_by`, `period`, `area`, `well_type`
`repairs`	read	Maintenance jobs that started in a period, with cost per well and totals.	`wells`, `area`, `period`, `maintenance_type`, `equipment_kind`
`rca_due`	read	Wells needing a root-cause analysis: corrective repairs on the same equipment kind in the last 12 months at or above the threshold (manual §10, or a team rule if one was recorded).	`area`, `equipment_kind`, `well_type`, `wells`
`compliance`	read	Rule breaches and open items vs the SOPs for a well or area: SR-01 report deadline, SR-02 containment (from repair reports), SR-03 clean-up verification, RC-04 notice responses, RC-02 flowline test, RC-01 inspection gaps, pending RCAs, unrecorded shut-ins.	`well`, `area`, `period`, `protocols`, `only_breaches`

`inspection_detail`	read	One inspection: readings, AI spill estimate, corrections, notes that agree or disagree, repairs and regulator events it triggered, and whether a spill detection stands.	`inspection_id`, `well`, `on_date`
`wells`	read	List wells with location and producing status at the latest inspection (for maps).	`area`, `well_type`, `status`
`search_documents`	read	Search manuals, SOPs and repair reports for rules, deadlines and facts that only appear in text.	`query`, `kind`, `well`
`query_sql`	read	Read-only SQL (SQLite) for questions no other tool covers. Tables: wells, equipment, areas, teams, users, inspections, maintenance, regulatory_events, notes, corrections, report_compliance, well_type_rules, rule_overrides, claims; views inspection_facts (corrected readings, temp_delta_c, producing, eff_spill, eff_spill_volume_bbl, area_name) and latest_inspection.	`sql`
`recall_my_conversations`	read	This user's own earlier conversations (never anyone else's). Use for 'the wells we discussed yesterday'.	`days_ago`
`record_correction`	write	Save a user's correction to a raw reading or to the AI's analysis of one inspection. Keeps the original. field: wellhead_temp_c | ambient_temp_c | spill_detected | spill_est_volume_bbl. kind: raw_data | ai_analysis.	`inspection_id`, `well`, `on_date`, `field`, `corrected_value`, `reason`, `kind`
`record_note`	write	Save shared knowledge about a well that every user should see (planned shut-ins, workovers, context). Use scope well_type with well_type for notes about a whole type.	`well`, `text`, `valid_from`, `valid_to`, `scope`, `well_type`
`record_rule`	write	Save a team rule that replaces a manual threshold for one well type only. rule: rca_min_corrective | producing_min_delta_c.	`rule`, `value`, `well_type`, `equipment_kind`, `statement`
`record_claim`	write	Save and fact-check a user's statement about whether a well is running or stopped now. claimed_state: producing | shut_in | down. Compares with the latest inspection and with what others said.	`well`, `claimed_state`, `since`, `statement`

Agent action protocol

```
{"thought": "short plan", "calls": [{"tool": "downtime", "args": {"area": "Northeast Block 7", "period":
"last_4_months", "top_n": 3}}]}
{"thought": "…", "final": {"answer": "markdown with [INS-…] citations",
                           "visual": {"step": 1, "kind": "bar", "title": "Downtime, last 4 months"},
                           "citations": ["INS-0121-0201", "MNT-0121-030"]}}
```

Visual selection

Answer is about	Kind	Rendered from

one well's state	status_card	card of well_status / inspection_detail / record_correction
a list	table	rows
a value over time	line	rows(date, wellhead_temp_c, air_temp_c, producing)
comparing wells / areas	bar	rows + hint{x, y}
where things are	map	rows with lat/lon (looked up by well if missing)
two dimensions	heatmap	aggregate(rows_by, columns_by) matrix + total

A visual named in the question (regex in `requested_visual()`) overrides the model's choice.

7. Memory design

Level	Write path	Read path	Scope rule
Shared notes	record_note → notes(origin=copilot)	well_status._current_notes, reasons, downtime causes	well / well_type / fleet
Corrections	record_correction → corrections	inspection_facts view (all tools)	one inspection, every user
Team rules	record_rule → rule_overrides	rca_due, inspection_facts threshold	one well type (or fleet), nothing else
Claims	record_claim → claims + notes	well_status.user_claims, other claims	one well; status from data
Private memory	services persist messages with asset_ids	recall_my_conversations (user filter, excludes current chat)	one user only
Answer ratings	rate_answer → answer_feedback	quality review	own answers only

Authorship is never taken from the model: Feedback is constructed with the signed-in user id. Superseded corrections and rules stay in the table with status `superseded`.

8. Guardrails

Layer	Check	Where
Input	empty / > 2,000 chars; prompt-injection patterns; delete/overwrite intent	guardrails.check_input
Prompt	tools-only facts, cite only tool IDs, never invent causes, stay on topic, do not reveal instructions	prompts.SYSTEM
Tool args	required args, known well/area/user, valid period, field and rule enums	ToolBox.run, Facts._scope_filter, Feedback
SQL	sqlglot AST: one SELECT, no DML/DDL/PRAGMA, allowlisted tables (no chat tables), LIMIT 200, read-only connection	sql_guard.guard_sql, session.get_read_only_engine
Output	unknown citations removed; ungrounded numbers flagged; requested visual enforced; tool-first for factual questions	Agent._finish
Privacy	conversation ownership checked on ask / read / rate	services.copilot

Resilience	model fallback 26B→31B with backoff; budget exhaustion returns the best tool summary, marked degraded	llm.GemmaOnGemini, Agent.run

9. Configuration, quality and operations

Setting	Default
`LLM_PROVIDER`	gemini (or ollama)
`GEMMA_MODEL / FALLBACK`	gemma-4-26b-a4b-it / gemma-4-31b-it
`OLLAMA_MODEL`	gemma4:e4b
`DATABASE_URL`	sqlite:///./data/wellwell.db
`TODAY`	2026-10-17
`AGENT_MAX_STEPS · timeouts`	6 rounds · 120 s · 40 s per call

- **Unit tests** (pytest): periods, spill classes, SQL guard, JSON parser, guardrails, manual and report extractors.
- **Scenario eval**: `eval/run_examples.py` replays the 18 example scenarios incl. multi-user sequences; checks facts and visual; writes `eval/last_run.json`.
- **Audit trail**: tool_runs keeps every call's args, result, error and latency.
- **Cost**: Gemma 4 on the Gemini API free tier; 2–3 model calls per question; deterministic layer in milliseconds.
- **Scale path**: same models on Postgres; read-only role for agent SQL; row-level security per team.